Summer Internship

LLM USING PYTHON

(Summer Internship 2026)

Presented by: -

Ramrup Satpati

B.Tech.(CSE), 5th Semester

SiC: 24BCSA05

Table of Contents

1 Introduction 2

2 Objectives 3

3 Contents of the Program 3

3.1 Python and LLM Fundamentals 4

3.2 Prompt Engineering and Model Interaction 4

3.3 Backend Devlopment IAM Platform Project 5

4 Methodology 6

5 Details of Works done during the Internship (on daily/weekly basis) 7

6 Outcomes of the program 15

7 Conclusions 16

8 Acknowledgements 17

1 Introduction

The Summer Internship Program titled “LLM with Python” was undertaken at Silicon University, Bhubaneswar, under the supervision of Mr. Sourav Mishra, from 20 May to 23 June 2026. The program focused on understanding Large Language Models (LLMs), learning the Python tools used to work with them, and exploring practical applications of generative AI.

The program began with Python concepts and the libraries commonly used in machine learning and natural language processing. It then progressed to text preprocessing, tokenization, embeddings, transformer architecture, and the working principles of large language models. Further learning covered prompt engineering, model inference, and the use of Python libraries and APIs to interact with language models. These topics were approached through conceptual study and practical exercises to build an understanding of how LLM-based applications are developed.

After establishing the foundations, the program progressed to building blocks of LLM applications. The learning covered preparing text for model input, generating and using embeddings, constructing effective prompts, and obtaining responses from language models through Python. The later part of the program introduced application patterns such as document-based question answering and Retrieval-Augmented Generation (RAG), in which relevant information is retrieved before a model generates an answer. The program therefore connected the fundamentals of language models with practical Python-based generative AI workflows.

2 Objectives

- To understand the fundamentals of Large Language Models and their role in generative artificial intelligence.
- To strengthen Python programming skills and become familiar with libraries used in machine learning and natural language processing.
- To study text preprocessing, tokenization, embeddings, and the basic components of transformer-based models.
- To understand prompt engineering, model inference, and the factors that influence generated responses.
- To use Python libraries and APIs to interact with language models and develop practical exercises.
- To understand Retrieval-Augmented Generation (RAG) and the role of document retrieval in LLM applications.
- To explore how embeddings and vector search can support semantic retrieval from text collections.
- To connect the concepts studied in the program through the design of a Python-based LLM application.

3 Contents of the Program

The program covered Python foundations for AI, natural language processing, the fundamentals of transformer-based language models, prompt engineering, and practical

LLM application development. The learning path moved from foundational concepts to Python-based model interaction and application patterns such as embeddings, semantic search, and Retrieval-Augmented Generation. The following subsections summarize the main areas covered.

3.1 Python and LLM Fundamentals

The foundational part covered Python programming concepts and the role of Python libraries in machine learning and natural language processing. The study then introduced the structure and purpose of Large Language Models, including how text is represented as tokens and how tokenization converts text into model-readable units. Embeddings were covered as numerical representations of text that can capture semantic relationships. The program also introduced the transformer architecture and the role of attention mechanisms in processing contextual relationships between tokens.

3.2 Prompt Engineering and Model Interaction

The practical learning phase focused on communicating with language models through carefully designed prompts. Prompt engineering involved specifying the task, providing relevant context, defining constraints, and structuring the expected output. Python was used as the interface for preparing inputs, sending requests to language models, and processing generated responses. The exercises highlighted that output quality depends on prompt clarity, context, and the capabilities and limitations of the selected model.

3.3 LLM Application Development with Python

The application-development phase explored how individual LLM capabilities can be combined into useful Python workflows. Topics included generating embeddings for text, retrieving semantically relevant passages, and supplying those passages as context to a language model. Retrieval-Augmented Generation (RAG) was introduced as a pattern that combines information retrieval with text generation, helping an application answer questions using a supplied document collection. The application topics also included organizing the workflow into stages for document preparation, retrieval, prompt

construction, model inference, and response handling. These concepts provide a practical foundation for developing Python-based applications powered by LLMs.

4 Methodology

The internship followed a progressive, practice-oriented learning approach. First, Python foundations and the role of commonly used AI and NLP libraries were reviewed through small exercises. The learning then moved to text preprocessing, tokenization, embeddings, and transformer-based language models. Conceptual study was combined with practical exploration to understand how text is prepared and processed by an LLM.

The next stage focused on prompt design and interacting with language models through Python. The final stage introduced application-level workflows, particularly semantic retrieval and Retrieval-Augmented Generation. The overall methodology combined topic-wise study, experimentation with model inputs and outputs, and the integration of concepts into a structured LLM application workflow.

5 WEEK 1 — PYTHON FOUNDATIONS AND INTRODUCTION TO LLMS

5.1 Learning focus

The first week focused on establishing a working foundation in Python for AI-related tasks and understanding the purpose of Large Language Models. The learning sequence covered Python data types, variables, control flow, functions, modules, and basic data handling. These concepts form the base for writing programs that prepare data, call model interfaces, and process generated results. The week also introduced generative AI and the kinds of language tasks that LLMs can perform, such as summarization, question answering, and text generation.

5.2 Python programming and AI libraries

Practice with Python functions, collections, modules, and exception handling helped establish the programming skills needed for LLM workflows. The learning also introduced the role of libraries used in data processing and natural language processing. Python can be used to clean and organize text, prepare structured inputs, communicate with model interfaces, and post-process model outputs. Emphasis was placed on writing modular code so that data preparation, model interaction, and response handling can be developed and tested separately.

5.3 Understanding Large Language Models

The week introduced Large Language Models as machine learning models trained on large text datasets to learn patterns in language and generate contextually relevant text. The study considered common use cases such as text completion, summarization, translation, and conversational question answering. It also introduced important limitations: an LLM may generate plausible but incorrect information, its response depends on the context supplied, and its knowledge may not include recent or private information. These limitations make evaluation and careful use important when building applications.

5.4 Text representation and tokenization

Text representation was studied as a key step in language-model processing. Tokenization divides text into smaller units called tokens, which may correspond to words, subwords, or characters depending on the tokenizer. Models process token IDs rather than raw text, and the number of tokens affects input length and computational cost. Understanding tokenization helps explain context-window limits, prompt length, and why the same text may be divided differently by different models.

6 WEEK 2 — NLP, EMBEDDINGS AND TRANSFORMER ARCHITECTURE

6.1 Text preprocessing and embeddings

The second week continued with natural language processing concepts used before and alongside LLM inference. Text preprocessing can include normalization, cleaning, sentence segmentation, and handling unwanted or inconsistent content. The study then covered embeddings, which represent words, sentences, or passages as numerical vectors. Texts with related meanings can have nearby vector representations, making embeddings useful for semantic similarity and information retrieval. This provides a foundation for searching text by meaning rather than relying only on exact keyword matches.

6.2 Transformer architecture and attention

The program introduced the transformer architecture, which is central to many modern language models. Attention mechanisms allow a model to weigh relationships between tokens when building contextual representations. This helps a model interpret a word in relation to the surrounding text rather than treating every word independently. The study also introduced the high-level distinction between model training and inference: training adjusts model parameters from data, while inference uses an already trained model to produce outputs for a given input.

6.3 Model families, context windows, and inference

The learning covered the practical meaning of model inputs, outputs, and context windows. A context window limits the amount of tokenized information a model can consider in a single interaction. Inference involves supplying a prompt and receiving generated tokens, with generation affected by model configuration and decoding settings. The program also considered that different models can vary in capability, response quality, context length, latency, and computational requirements. These factors influence the choice of model for a particular application.

6.4 Practical exercise: text and embedding workflow

A practical exercise connected text preparation with semantic representation. The workflow involved preparing text, converting it into embeddings, and considering how vector similarity can identify related pieces of content. This exercise provided a foundation for retrieval-based applications. It also demonstrated why consistent preprocessing and an appropriate embedding model matter when comparing text passages.

7 WEEK 3 — PROMPT ENGINEERING AND PYTHON MODEL INTEGRATION

7.1 Interacting with language models

The third week shifted from understanding model concepts to interacting with language models through Python. The focus was on the structure of a model request: preparing an input, supplying instructions and context, invoking the model, and handling the generated response. Python provides a way to integrate model calls into scripts and applications. The phase connected language-model concepts to practical software workflows, where inputs, outputs, errors, and application requirements need to be handled consistently.

7.2 Prompt structure and response handling

Prompt engineering considered how instructions, context, examples, and output requirements influence a model response. A well-structured prompt clearly defines the task and provides only the context needed to complete it. Where a predictable response format is required, output constraints or structured formats can make results easier to process. The Python application must still validate and handle the response because generated text should not automatically be treated as correct or trusted data.

7.3 Python libraries and model APIs

The program explored the role of Python libraries and model APIs in connecting an application to a language model. A typical workflow prepares the prompt, sends a request using the selected interface, receives the response, and processes it for the application. API-based access may require configuration and secure handling of credentials. Practical integration also requires handling network failures, rate limits, invalid inputs, and model errors. These considerations help make model interaction more reliable.

7.4 Responsible and effective LLM application design

The transition to application development highlighted several design concerns. Model outputs can be inaccurate, incomplete, or inconsistent, so applications should validate outputs and make uncertainty clear where appropriate. Sensitive information should not be included in prompts unless the application and service are approved to process it. Developers should also consider latency, token usage, cost, context limits, and the need for human review in consequential workflows. These principles prepared the ground for building a more complete LLM-based application.

8 WEEK 4 — PYTHON-BASED LLM APPLICATION AND RAG

8.1 Application workflow and document preparation

The final project phase brought together the concepts studied during the program through the design of a Python-based LLM application. A document-oriented question-answering workflow provides a useful application context: source documents are collected, text is extracted, and the content is divided into manageable chunks. Chunking helps keep retrieved context within model limits while retaining enough surrounding information to make passages useful. The prepared text can then be converted into embeddings for semantic retrieval.

8.2 Semantic retrieval and vector search

The application workflow used embeddings as the basis for semantic retrieval. A user query can be converted into an embedding and compared with stored document embeddings to identify relevant passages. Similarity search helps retrieve content that is conceptually related to the query, even when the wording differs. The retrieved passages should be checked for relevance and then supplied to the language model as supporting context. Retrieval quality is important because irrelevant or incomplete context can lead to weak or unsupported answers.

8.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines a retrieval step with language-model generation. Instead of relying only on information encoded during model training, the application retrieves relevant content from an external collection and includes it in the prompt. The model then generates a response grounded in the supplied context. This pattern is useful for document question answering and knowledge-base applications. It does not guarantee correctness: the retrieved material must be relevant, and the generated answer should be evaluated against the source content.

8.4 Evaluation, limitations, and responsible use

The final stage considered how to assess an LLM application beyond whether it produces fluent text. Useful checks include whether retrieved passages are relevant, whether the response answers the question, and whether its claims are supported by the supplied documents. Common limitations include hallucinations, retrieval failures, ambiguous prompts, context-window constraints, and changes in model behaviour. The application workflow therefore benefits from testing with representative questions, clear prompts, output validation, and appropriate human oversight. Overall, the program connected Python programming, NLP, model interaction, embeddings, and RAG in a practical LLM application context.

9 Outcomes of the program

The program provided structured exposure to Large Language Models and their use through Python, progressing from programming foundations to NLP concepts and practical generative AI workflows. The topics studied built an understanding of tokenization, embeddings, transformer architecture, attention, model inference, prompt engineering, and the limitations of generated outputs.

The application phase connected these concepts through Python-based model interaction and a document-oriented Retrieval-Augmented Generation workflow. The program introduced document preparation, chunking, semantic retrieval, context construction, and response generation. It also highlighted the importance of evaluating retrieved information and model responses, handling errors, and using LLMs responsibly. The program connected conceptual learning with practical software development and provided a foundation for further work in generative AI and LLM application engineering.

10 Conclusions

The “LLM with Python” internship provided a progression from Python foundations to the concepts and application patterns used in modern generative AI. Studying text preprocessing, tokenization, embeddings, transformer architecture, and attention helped establish an understanding of how language models process text. Prompt engineering and Python-based model interaction then introduced the practical steps involved in using an LLM within an application.

The final application phase connected these topics through semantic retrieval and Retrieval-Augmented Generation. It highlighted how external documents can provide context for model responses, while also emphasizing the need to evaluate outputs, handle limitations, and use models responsibly. Overall, the program strengthened the connection between Python programming, natural language processing, and LLM application design. Further experimentation and testing can deepen understanding of reliable, production-oriented generative AI systems.

11 Acknowledgements

I express my sincere gratitude to Silicon University, Bhubaneswar, for providing the opportunity to undertake the “LLM with Python” Summer Internship Program. I am thankful to my supervisor, Mr. Sourav Mishra, for his guidance and support throughout the program. I also acknowledge the learning resources and practical exercises that helped me study Python foundations, explore natural language processing and large language models, and understand practical workflows such as prompt engineering, semantic retrieval, and Retrieval-Augmented Generation. This program has been a valuable part of my technical learning and development.